DLDCA Lab 1: Verilog

Week 1

August 7, 2026

Introduction

In this lab we are going to start with a simple set of problems through which we will understand the basics of verilog syntax and semantics. The lab consists of three tasks each with the goal of teaching combinational circuits, along with an additional graded task.

Necessary tools

We shall be using `icarus-verilog` (or) `iverilog` in order to compile verilog HDL (Hardware Description Language). To visualize waveforms we shall use the VS Code extension 'Vaporview'. Before starting the lab, ensure that both tools are installed in your system. This can be verified by running the following:

```
$ iverilog
iverilog: no source files.

Usage: iverilog [-EiRSuvV] [-B base] [-c cmdfile|-f cmdfile]
                [-g1995|-g2001|-g2005|-g2005-sv|-g2009|-g2012] [-g<feature>]
                [-D macro[=defn]] [-I includedir] [-L moduledir]
                [-M [mode=]depfile] [-m module]
                [-N file] [-o filename] [-p flag=value]
                [-s topmodule] [-t target] [-T min|typ|max]
                [-W class] [-y dir] [-Y suf] [-l file] source_file(s)

See the man page for details.
```

Structure of submission/templates

Submission format:

```
your-roll-no/
|- task0/
    |- and_gate.v (TODO)
    |- tb_task0.v
|- task1/
    |- adder.v (TODO)
    |- tb_task1.v
|- task2/
    |- adder.v
    |- fourbit_adder.v (TODO)
    |- tb_task2.v
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of `your-roll-no.tar.gz`. The command given below can be used to do the same

```
$ ./build_archive.sh
Enter Student Roll Number: your-roll-no
Packaging assignment files...
  [+] Copied: ...
Successfully created your-roll-no.tar.gz
```

Tasks

Task 0: Hello Verilog!

This task is a warm up to get you familiar with the basic Verilog workflow: writing a module, simulating it against a testbench, and viewing the resulting waveforms.

The goal is to implement a simple 2-input AND gate. The module takes two 1-bit inputs, `in1` and `in2`, and produces a single 1-bit output `out` that is high only when both inputs are high.

You have been provided a template module file (`and_gate.v`) with the port list already declared but the body left empty. Fill in the logic so that `out` is continuously assigned the bitwise AND of `in1` and `in2`.

A testbench (`tb_task0.v`) has also been provided. It instantiates your module, drives all four combinations of `in1` and `in2`, and prints the values of the inputs and output at every step using `$monitor`. It also dumps the simulation to a `.vcd` file so you can inspect the waveform.

To run your code, use the makefile:

```
$ make task0
```

This will open `Vaporview` within VS Code where you can see the output. Make sure to go through the testbench code to understand how `$dumpfile` and `$dumpvars` calls work.

Task 1: Adding Bits

The goal of this task is to create a simple one bit adder module that takes two input bits (a, b), and a carry in bit to output a carry out bit and an sum bit. The architecture for the 1 bit adder can be seen in figure 1.

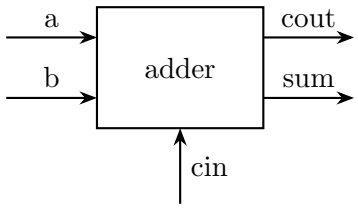


Figure 1: Adder architecture

The truth table for the circuit is as follows

a	b	cin	cout	sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Derive the formulae for the `sum` and `cout` wires from the truth table using karnaugh maps.

You have been provided a template module file (`module.v`), you are expected to fill in the logic to complete the adder. In order to test your implementation we have also provided you a testbench (`tb_module.v`).

To run your code, use the makefile:

```
$ make task1
```

This will open a waveform viewer within VSC where you can see the output

Make sure to go through the testbench code to understand how to output your testbench output to `.vcd` files in order to visualize them with the extension.

Task 2: Scaling Up

Having built a 1-bit adder in Task 1, we can now use it as a building block to construct a wider adder. This task asks you to build a 4-bit adder by chaining together four copies of your 1-bit `adder` module, connecting the carry-out of each stage to the carry-in of the next. This structure is called a **ripple-carry adder**, since the carry "ripples" from the least significant bit to the most significant bit.

The 4-bit adder takes two 4-bit inputs `A` and `B`, along with a single carry-in bit `Cin`, and produces a 4-bit sum `Sum` and a single carry-out bit `Cout`. `A[0]/B[0]` are the least significant bits and `A[3]/B[3]` are the most significant bits.

Your completed `adder.v` from Task 1 has been copied into this folder, since you will need to instantiate it four times inside `4bit_adder.v`. Each instance computes one bit position; the carry-out of instance i should be wired to the carry-in of instance $i + 1$, with `Cin` feeding the first instance and the last instance's carry-out becoming `Cout`.

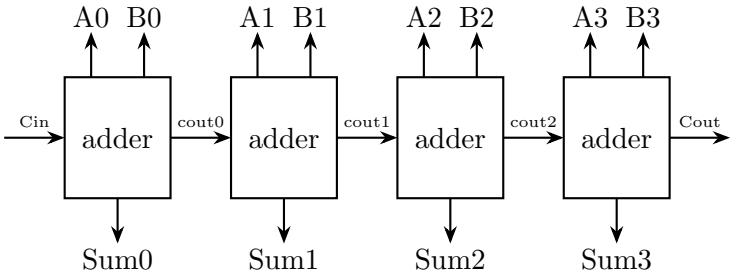


Figure 2: 4-bit ripple-carry adder built from four 1-bit adders

Notice that unlike Tasks 0 and 1, the ports here are declared as *vectors* (`[3:0]`) rather than single bits. You can index into a vector using `signal[i]` to access an individual bit, which is how you will connect each 1-bit adder instance to the right bit of `A`, `B`, and `Sum`.

A testbench (`tb_task2.v`) has been provided, which drives a handful of directed test cases – including ones that produce a carry out of the top bit – and prints the inputs and outputs at each step, while also dumping a waveform for you to inspect.

To run your code, use the makefile:

```
$ make task2
```

This will open a waveform viewer within VSC where you can see the output.